

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK (PBO)

***MEMBUAT FUNGSI CRUD (Create, Read, Update, Delete)
USER DENGAN DATABASE MYSQL***



OLEH :

NIA RAMADHANI
2311531006

DOSEN PENGUMPU : Nurfiah, S.ST, M.Kom.

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS**

1.1 Pendahuluan

Pada praktikum ini, saya diharapkan mampu mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk data pengguna menggunakan database MySQL. Praktikum ini bertujuan untuk melatih kemampuan saya dalam membuat tabel pengguna di MySQL, menghubungkan aplikasi Java dengan database, membuat tampilan GUI (Graphical User Interface) untuk CRUD pengguna, serta mengaplikasikan konsep Pemrograman Berorientasi Objek (PBO). Selain itu, saya juga akan mempelajari pembuatan dan penggunaan interface serta fungsi DAO (Data Access Object) yang dapat memisahkan logika akses data dari logika bisnis, meningkatkan modularitas, dan memungkinkan perubahan dalam pengelolaan data tanpa mempengaruhi bagian lain dari aplikasi. Alat-alat yang dibutuhkan untuk praktikum ini antara lain komputer yang telah terinstal JDK, Eclipse, MySQL atau XAMPP, serta MySQL Connector untuk menghubungkan Java dengan MySQL.

1.2 Tujuan

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

- Mahasiswa mampu membuat table user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikannya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

1.3 Alat

- Computer / laptop yang telah terinstall JDK dan Eclipse
- MySQL / XAMPP
- MySQL connector atau Connector/J

1.4 Teori

XAMPP yaitu paket software yang terdiri dari Apache HTTP Server, MySQL, PHP dan Perl yang bersifat open source, xampp biasanya digunakan sebagai development environment dalam pengembangan aplikasi berbasis web secara localhost.

Apache berfungsi sebagai web server yang digunakan untuk menjalankan halaman web, MySQL digunakan untuk manajemen basis data dalam melakukan manipulasi data, PHP

digunakan sebagai Bahasa pemrograman untuk membuat aplikasi berbasis web.

MySQL adalah sebuah relational database management system (RDBMS) open-source yang digunakan dalam pengelolaan database suatu aplikasi, MySQL ini dapat digunakan untuk menyimpan, mengelola dan mengambil data dalam format table.



Logo MySQL

MySQL Connection/j adalah driver yang digunakan untuk menghubungkan aplikasi berbasis java dengan database MySQL sehingga dapat berinteraksi seperti menyimpan, mengubah, mengambil dan menghapus data. Beberapa fungsi MySQL connector yaitu :

- Membuka koneksi ke database MySQL
- Mengirimkan permintaan SQL ke server MySQL
- Menerima hasil dari permintaan SQL
- Menutup koneksi ke database MySQL

DAO (Data Access Object) merupakan object yang menyediakan abstract interface terhadap beberapa method yang berhubungan dengan database seperti mengambil data (read), menyimpan data(create), menghapus data (delete), mengubah data(update). Tujuan penggunaan DAO yaitu :

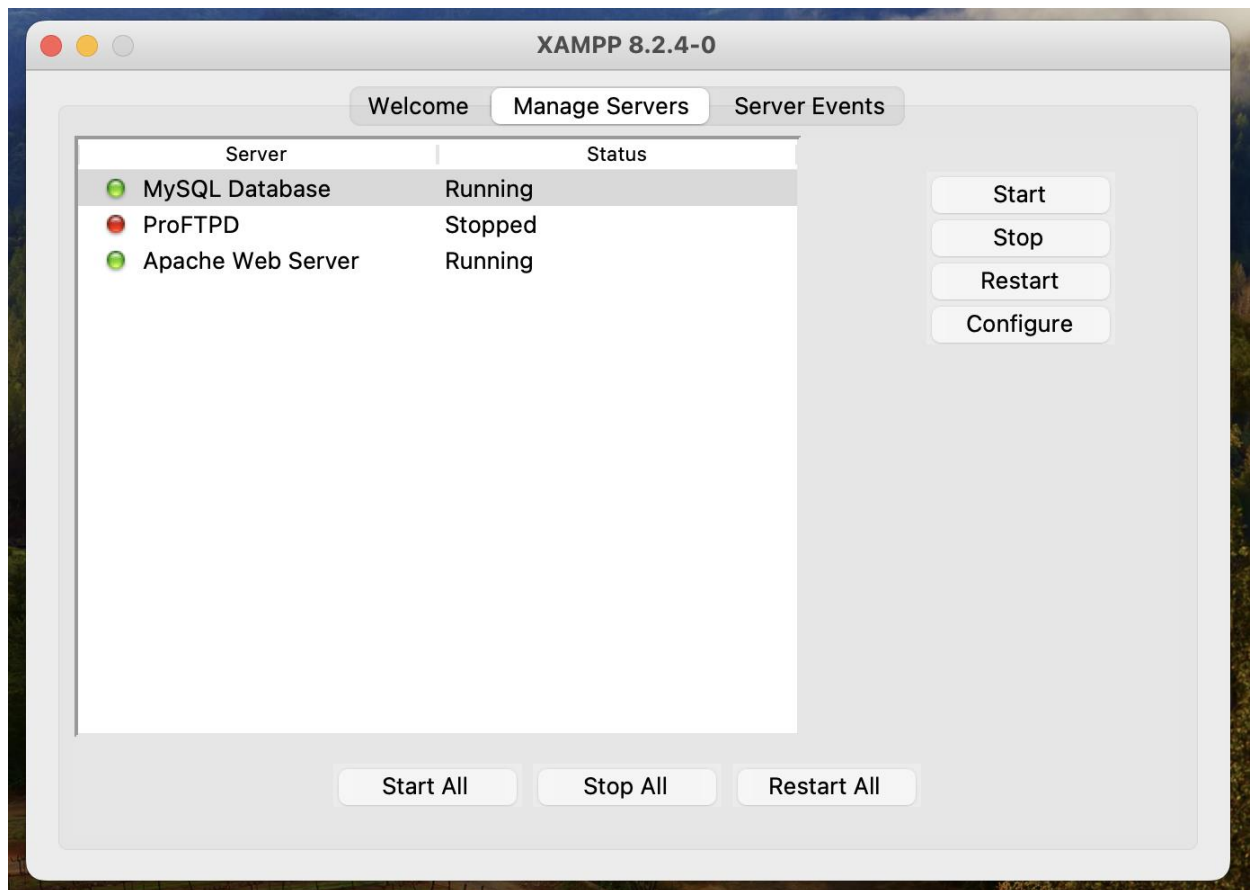
- Meningkatkan modularitas yaitu memisahkan logika akses data dengan logika bisnis sehingga memudahkan untuk dikelola
- Meningkatkan reusabilitas yaitu DAO dapat digunakan Kembali
- Perubahan pada logika akses data dapat dilakukan tanpa mempengaruhi logika bisnis.

Interface dalam Bahasa java yaitu mendefinisikan beberapa method abstrak yang harus diimplementasikan oleh class yang akan menggunakannya.

CRUD (Create, Read, Update, Delete) merupakan fungsi dasar atau umum yang ada pada sebuah aplikasi yang mana fungsi ini dapat membuat, membaca, mengubah dan menghapus suatu data pada database aplikasi.

1.5 Langkah-langkah Install XAMPP

- Download XAMPP dari pada link berikut : <https://www.apachefriends.org/>
- Setelah didownload install xampp pada computer/laptop masing-masing
- Jalankan xampp dan aktifkan apache dan mysql



Menambahkan MySQL Connector

Aplikasi java agar dapat terhubung dengan database MySQL membutuhkan sebuah driver yaitu MySQL Connection, berikut ini Langkah-langkah membuat koneksi Database MySQL.

- Download MySQL connection pada link berikut
<https://dev.mysql.com/downloads/connector/j/>

MySQL Community Downloads

Connector/J

General Availability (GA) Releases Archives ⓘ

Connector/J 9.0.0

Select Operating System:

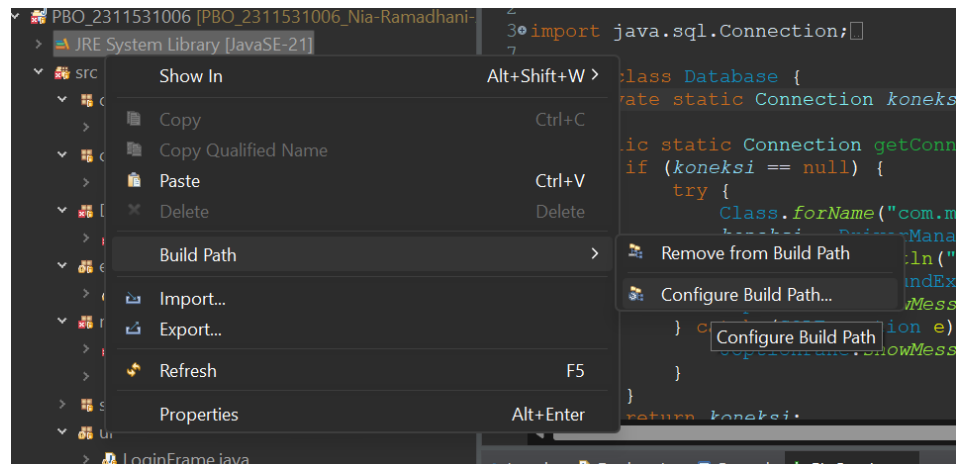
Platform Independent ⓘ

Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-9.0.0.tar.gz)	9.0.0	4.3M	Download
		MD5: 820b4d2fa1108130617093a444ee1496 Signature	
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-9.0.0.zip)	9.0.0	5.1M	Download
		MD5: aeaf0db3a50f8756e58eb7a6aa21777d Signature	

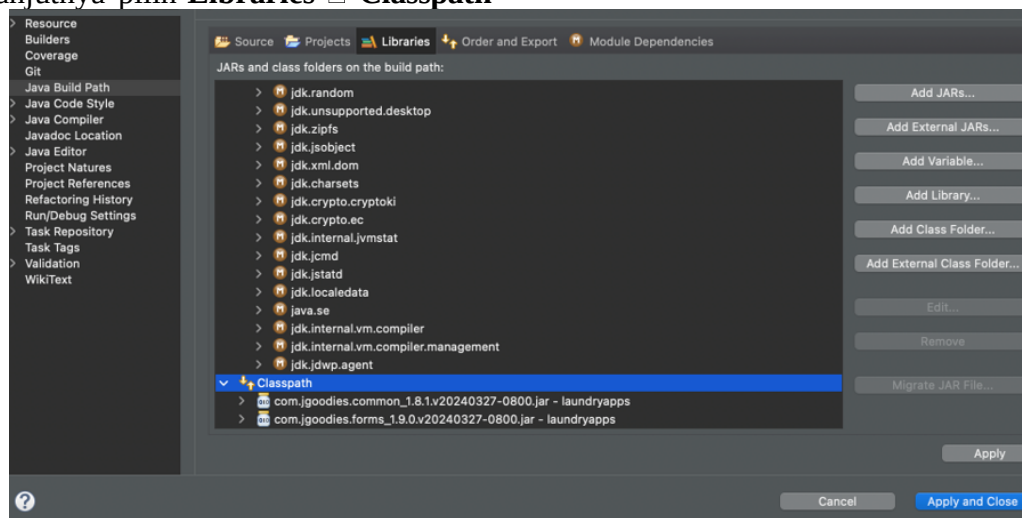
 We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Pilih file yang berekstensi .zip

- Menambahkan MySQL Connector kedalam project dengan cara klik kanan directory **JRE System Library** ☐ **Built Path** ☐ **Configure Build Path**



Selanjutnya pilih **Libraries** ☐ **Classpath**



Tambahkan file MySQL Connector dengan cara klik **Add External JARs**

dan pilih file yang telah didownload dan pilih **Apply and Close**.

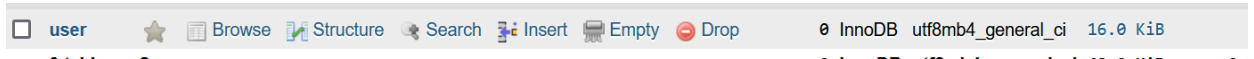
Jika berhasil menambahkan MySQL Connector maka akan generate folder Referenced Libraries pada project yang berisi MySQL Connector.



Membuat Database dan Table User

- Buka <http://localhost/phpmyadmin>
- Klik new dan buat database dengan nama laundry_apps

- Buat table user dengan cara klik database laundry_apps dan buat table dengan nama user



Klik create maka akan muncul seperti gambar dibawah ini.

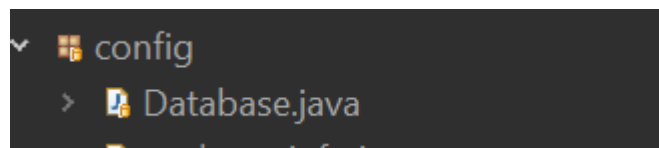
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None			Change Drop More
2	name	varchar(128)	utf8mb4_general_ci		No	None			Change Drop More
3	username	varchar(128)	utf8mb4_general_ci		No	None			Change Drop More
4	password	text	utf8mb4_general_ci		No	None			Change Drop More

Isi seperti gambar diatas dan klik save.

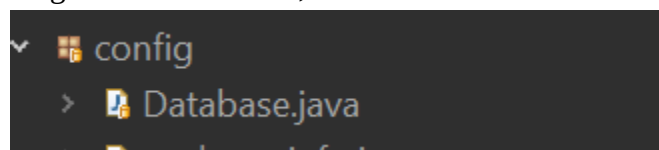
Membuat Koneksi ke Database MySQL

Setelah berhasil menambahkan MySQL Connector maka dapat membuat koneksi ke database MySQL, berikut Langkah-langkahnya.

- Buat package baru dengan nama config, package ini yang akan digunakan untuk membuat konfigurasi aplikasi yang akan dibuat termasuk dengan konfigurasi database.



- Buat class baru dengan nama Database,



kemudian konfigurasi sesuai dengan kode program beriku

```

1 package config;
2
3 import java.sql.Connection;
4
5
6
7
8 public class Database {
9     private static Connection koneksi;
10
11     public static Connection getConnection() {
12         if (koneksi == null) {
13             try {
14                 Class.forName("com.mysql.cj.jdbc.Driver");
15                 koneksi = DriverManager.getConnection("jdbc:mysql://localhost:3306/");
16                 System.out.println("Koneksi berhasil!");
17             } catch (ClassNotFoundException e) {
18                 JOptionPane.showMessageDialog(null, "Driver tidak ditemukan: " + e.getMessage());
19             } catch (SQLException e) {
20                 JOptionPane.showMessageDialog(null, "Koneksi gagal: " + e.getMessage());
21             }
22         }
23         return koneksi;
24     }
25 }

```

Membuat Tampilan CRUD User

- Buat file baru menggunakan JFrame pada package ui dengan nama UserFrame seperti gambar berikut ini.

The image shows a Java Swing window titled "UserFrame". It has a light gray background. There are three text input fields stacked vertically. The first field is labeled "Name", the second "Username", and the third "Password". Below these fields, there are four buttons arranged horizontally: "Save", "Update", "Delete", and "Cancel". Each button has a blue gradient and a 3D effect.

Keterangan :

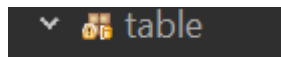
Component	Variable	Keterangan
JTextField	txtName	Name
JTextField	txtUsername	Username

JTextField	txtPassword	Password
JButton	btnSave	Save
JButton	btnUpdate	Update
JButton	btnDelete	Delete
JButton	btnCancel	Cancel
JTable	tableUsers	Table Users

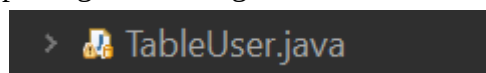
Membuat Table Model

Table model user ini berguna untuk mengambil data dari database dan ditampilkan kedalam table.

- Buat package baru dengan nama **table**



- Buat file baru didalam package table dengan nama **TableUser**,



kemudian isikan dengan kode program berikut.

 A screenshot of the Eclipse IDE showing the code for TableUser.java. The code is as follows:

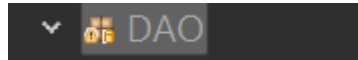
```

1 package table;
2
3 import java.util.List;
4
5 public class TableUser extends AbstractTableModel {
6     private List<User> userList;
7     private String[] columnNames = { "ID", "Name", "Username", "Password" };
8
9     public TableUser(List<User> userList) {
10         this.userList = userList;
11     }
12
13     @Override
14     public int getRowCount() {
15         return userList.size();
16     }
17
18     @Override
19     public int getColumnCount() {
20         return columnNames.length;
21     }
22
23     @Override
24     public Object getValueAt(int rowIndex, int columnIndex) {
25         User user = userList.get(rowIndex);
26         switch (columnIndex) {
27             case 0: return user.getId();
28             case 1: return user.getName();
29             case 2: return user.getUsername();
30             case 3: return user.getPassword();
31             default: return null;
32         }
33     }
34 }
  
```

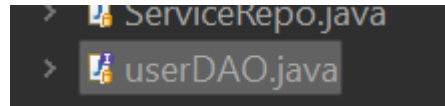
 The Package Explorer on the left shows the project structure with the 'table' package and 'TableUser.java' class. The Outline view on the right shows the class structure of TableUser.

Membuat Fungsi DAO

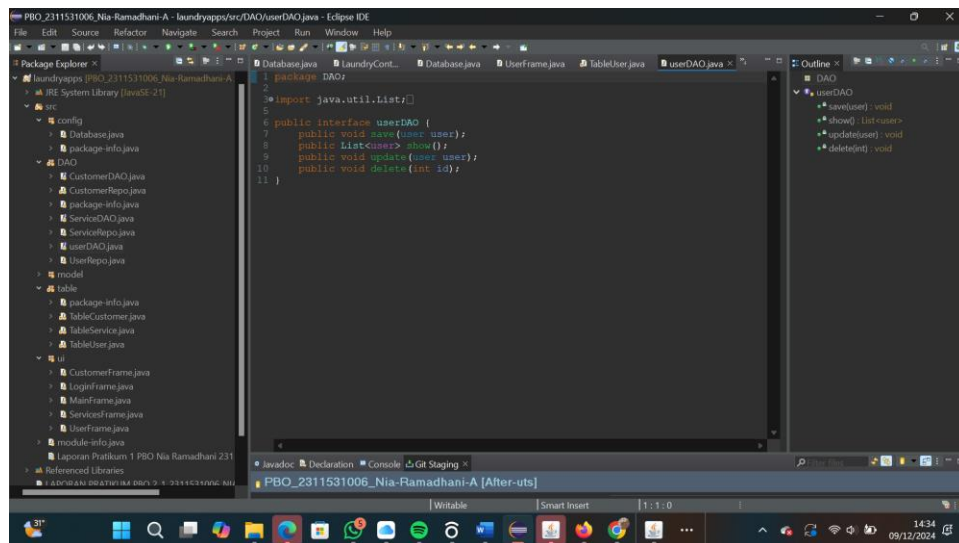
- Buat package baru dengan nama DAO



- Buat class Interface baru dengan nama **UserDAO**,



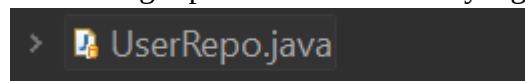
kemudian isikan dengan kode program berikut.



Terdapat method **save**, **show**, **delete** dan **update**. Method pada class interface digunakan sebagai method utama yang wajib diimplementasikan pada class yang menggunakannya.

Menggunakan Fungsi DAO

- Buat class baru pada package DAO dengan nama **UserRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.



- Implementasikan UserDao dengan kata kunci **implements**

```
public class UserRepo implements UserDAO {
```

- Masukkan kode di bawah ini

```
1 package DAO;
2
3 import java.util.ArrayList;
4
5
6
7 public class UserRepo {
8     private List<user> userList;
9
10    public UserRepo() {
11        userList = new ArrayList<>();
12    }
13
14    public List<user> show() {
15        return userList;
16    }
17
18    public void save(user user) {
19        user.setId(String.valueOf(userList.size() + 1)); // Simple ID generation
20        userList.add(user);
21    }
22
23    public void update(user user) {
24        for (user u : userList) {
25            if (u.getId().equals(user.getId())) {
26                u.setNama(user.getNama());
27                u.setUsername(user.getUsername());
28                u.setPassword(user.getPassword());
29                break;
30            }
31        }
32    }
33
34    public void delete(String id) {
```

- untuk method **save**, isikan dengan kode program berikut.

```
@Override
public void save(User user) {
    String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?)";
    try (PreparedStatement st = connection.prepareStatement(insert)) {
        st.setString(1, user.getNama());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

- Membuat method **show** untuk mengambil data dari database

```

@Override
public List<User> show() {
    List<User> users = new ArrayList<>();
    String select = "SELECT * FROM user";
    try (Statement st = connection.createStatement();
        ResultSet rs = st.executeQuery(select)) {
        while (rs.next()) {
            User user = new User();
            user.setId(rs.getString("id"));
            user.setName(rs.getString("name"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));
            users.add(user);
        }
    } catch (SQLException e) {
        Logger.getLogger(UserRepo.class.getName()).log(Level.SEVERE, null, e);
    }
    return users;
}

```

- Membuat method **update** yang digunakan untuk mengubah data.

```

@Override
public void update(User user) {
    String update = "UPDATE user SET name = ?, username = ?, password = ? WHERE id = ?";
    try (PreparedStatement st = connection.prepareStatement(update)) {
        st.setString(1, user.getName());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.setString(4, user.getId());
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

- Membuat method **delete** yang digunakan untuk menghapus data.

```

@Override
public void delete(String id) {
    String delete = "DELETE FROM user WHERE id = ?";
    try (PreparedStatement st = connection.prepareStatement(delete)) {
        st.setString(1, id);
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Menggunakan Fungsi CRUD DAO pada GUI

- method digunakan untuk menghapus value inputan Ketika suatu proses berhasil dilakukan, buat method **reset** pada JFrame seperti kode program dibawah ini.

```
public void reset() {  
    txtName.setText("");  
    txtUsername.setText("");  
    txtPassword.setText("");  
    selectedId = null;  
}
```

CREATE USER

- Klik kanan pada tombol **save** ☐ **add event handlers** ☐ **actionPerformed**

kemudian isi dengan kode program berikut.

```
User user = new User();  
user.setNama(txtName.getText());  
user.setUsername(txtUsername.getText());  
user.setPassword(txtPassword.getText());  
usr.save(user);  
reset();  
loadTable();  
JOptionPane.showMessageDialog(null, "User saved successfully!");
```

READ USERS

- Buat method dengan nama **loadTable()** kemudian isikan dengan kode program berikut.

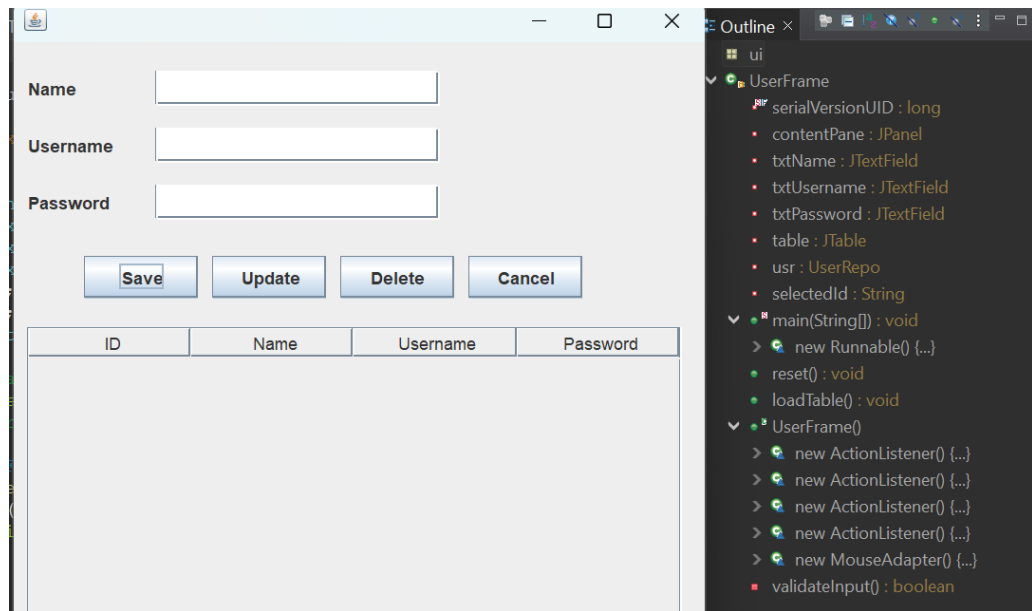
```
public void loadTable() {  
    List<User> ls = usr.show();  
    TableUser model = new TableUser(ls);  
    table.setModel(model);  
}
```

- Memanggil method pada class main, sehingga Ketika pertama kali program dijalankan maka **loadTable** akan dipanggil.

```
UserFrame frame = new UserFrame();  
frame.setVisible(true);
```

UPDATE USER

- Klik kanan pada **JTable** ☐ **add event handler** ☐ **mouse** ☐ **mouseClicked**



Isikan dengan kode program dibawah ini.

```
selectedId = table.getModel().getValueAt(row, 0).toString();
txtName.setText(table.getModel().getValueAt(row, 1).toString());
txtUsername.setText(table.getModel().getValueAt(row, 2).toString());
txtPassword.setText(table.getModel().getValueAt(row, 3).toString());
```


- Klik salah satu isi table maka akan secara otomatis tampil pada form inputan.

The screenshot shows a Java Swing window with a light gray background. At the top, there are three standard window control buttons (minimize, maximize, close). Below these, there are three text input fields labeled "Name", "Username", and "Password". Under the input fields, there are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a table with four columns: "ID", "Name", "Username", and "Password". The table contains two rows of data.

ID	Name	Username	Password
1	NANA	YAYA	OK
2	nisa	apa	iya

- Klik kanan tombol **update** ☐ **add event handler** ☐ **action** ☐ **actionPerformed** dan isikan dengan kode program berikut.

DELETE USER

- Klik salah satu data pada JTable
- Klik kanan tombol **delete** ☐ **add event handler** ☐ **action** ☐ **actionPerformed**

The screenshot shows a Java Swing window similar to the one above. The "Name" field contains "NANA", the "Username" field contains "YAYA", and the "Password" field contains "OK". The "Update" button is highlighted with a blue background. In the table below, the first row (ID 1, Name NANA, Username YAYA, Password OK) is selected, also with a blue background.

ID	Name	Username	Password
1	NANA	YAYA	OK
2	nisa	apa	iya

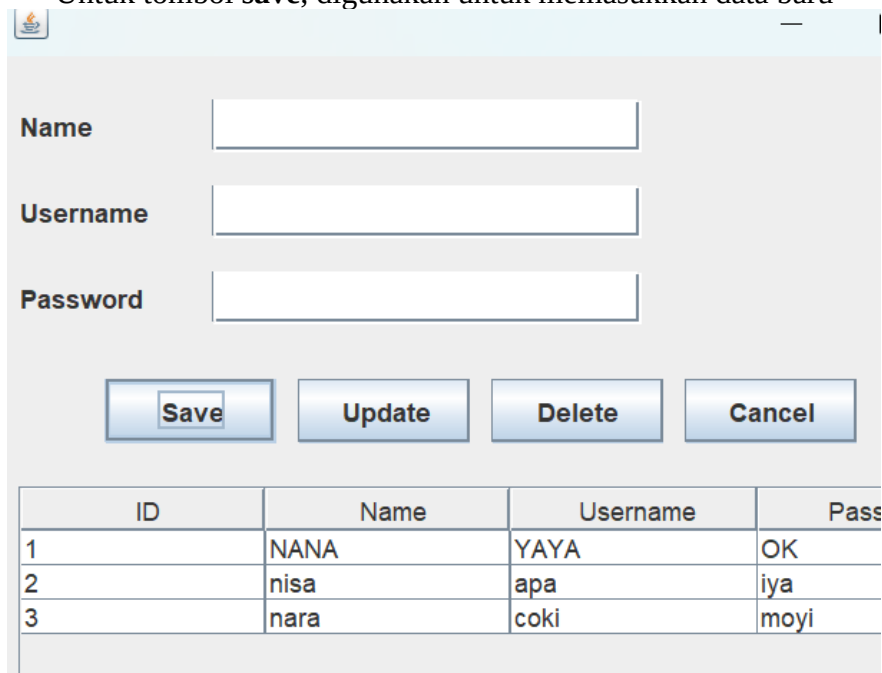
dan isikan dengan kode program berikut.

```

if (selectedId != null && validateInput()) {
    User user = new User();
    user.setId(selectedId);
    user.setName(txtName.getText());
    user.setUsername(txtUsername.getText());
    user.setPassword(txtPassword.getText());
    usr.update(user);
    reset();
    loadTable();
    JOptionPane.showMessageDialog(null, "User updated successfully!");
} else {
    JOptionPane.showMessageDialog(null, "Select a user first!");
}

```

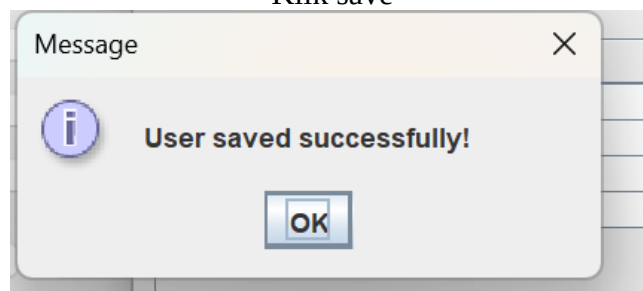
Untuk tombol **save**, digunakan untuk memasukkan data baru



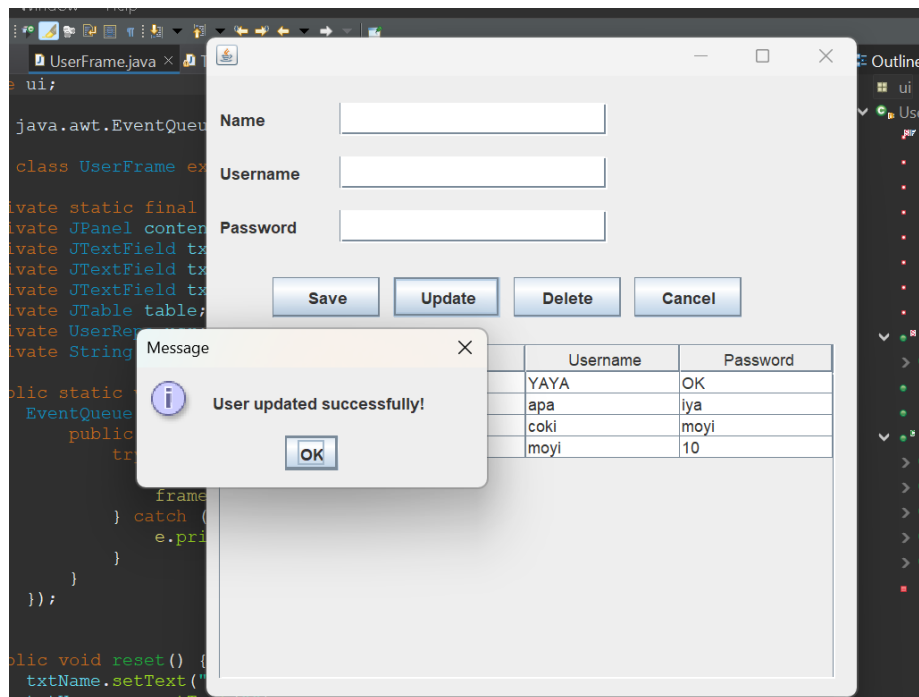
The screenshot shows a Java Swing application window titled "User Management". It contains a form with three text input fields labeled "Name", "Username", and "Password". Below the form are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a table with four columns: "ID", "Name", "Username", and "Password". The table contains three rows of data.

ID	Name	Username	Password
1	NANA	YAYA	OK
2	nisa	apa	iya
3	nara	coki	moyi

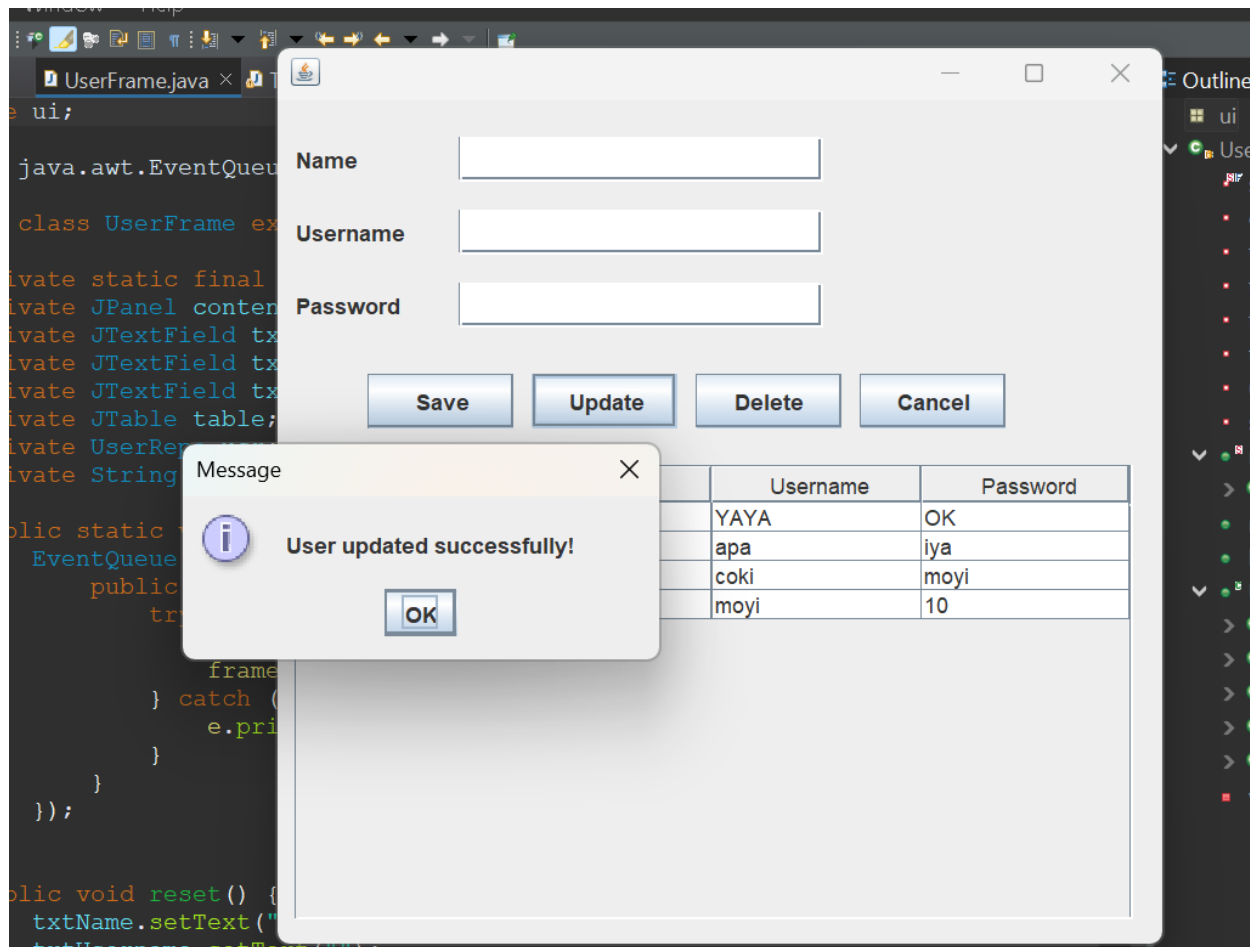
Klik save



Untuk tombol **update**, digunakan untuk memperbarui data



Klik update



1.6 Penutup

Melalui praktikum ini, saya telah mempelajari dan mempraktikkan pembuatan fungsi CRUD menggunakan Java dan MySQL dengan menerapkan konsep Pemrograman Berorientasi Objek. Saya juga telah berhasil membuat GUI untuk mengelola data pengguna serta memanfaatkan fungsi DAO untuk modularitas dan pemisahan logika. Dengan pemahaman yang didapatkan, saya diharapkan mampu mengimplementasikan konsep-konsep ini pada proyek-proyek pengembangan perangkat lunak di masa mendatang, terutama yang melibatkan pengelolaan data berbasis database.

